

2.4.1 Sorting Algorithms: Selection Sort

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 09 of 13

★ Golden

2.4.1 Sorting Algorithms: Selection Sort

LEARN IT

Selection Sort is a simple sorting algorithm that works by repeatedly **selecting the smallest element** from the unsorted portion of the list and moving it to its correct position at the front.

Think of it as filling seats from the front: first find who belongs in position 1, then position 2, and so on. In the textbook version, the element at position K is compared with every element after it; whenever a smaller one is found, the two are swapped. So at the end of pass K, the K-th smallest value sits in position K.

Fix position K

→

Compare DATA[K] with DATA[PTR], PTR = K+1 ... N

→

DATA[K] > DATA[PTR]? swap

→

End of pass: smallest is at K

→

K = K + 1

Algorithm (textbook)

Pseudocode

REMEMBER IT

Teacher's tip — **spot the difference:** In Selection Sort the inner loop starts at `outer + 1` and compares with the *fixed* position `DATA[outer]`. In Bubble Sort the inner loop always starts at 0 and compares neighbours `DATA[inner]` and `DATA[inner+1]`. Bubble pushes the largest to the **end**; Selection brings the smallest to the **front**.

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order (textbook method). The two positions in brackets are being compared.

Pass 1 — find the smallest for Position 1

Compare	List	Pos1 > PosK?	Action	List after
Pos1 vs Pos2	[8], [4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
Pos1 vs Pos3	[4], 8, [1], 9, 3	4 > 1 Yes	Swap	1, 8, 4, 9, 3
Pos1 vs Pos4	[1], 8, 4, [9], 3	1 > 9 No	No change	1, 8, 4, 9, 3
Pos1 vs Pos5	[1], 8, 4, 9, [3]	1 > 3 No	No change	1, 8, 4, 9, 3

1 is now permanently in Position 1.

Pass 2 — find the smallest for Position 2

Compare	List	Pos2 > PosK?	Action	List after
Pos2 vs Pos3	1, [8], [4], 9, 3	8 > 4 Yes	Swap	1, 4, 8, 9, 3
Pos2 vs Pos4	1, [4], 8, [9], 3	4 > 9 No	No change	1, 4, 8, 9, 3
Pos2 vs Pos5	1, [4], 8, 9, [3]	4 > 3 Yes	Swap	1, 3, 8, 9, 4

Pass 3 — find the smallest for Position 3

Compare	List	Pos3 > PosK?	Action	List after
Pos3 vs Pos4	1, 3, [8], [9], 4	8 > 9 No	No change	1, 3, 8, 9, 4
Pos3 vs Pos5	1, 3, [8], 9, [4]	8 > 4 Yes	Swap	1, 3, 4, 9, 8

Pass 4 — find the smallest for Position 4

Compare	List	Pos4 > PosK?	Action	List after
Pos4 vs Pos5	1, 3, 4, [9], [8]	9 > 8 Yes	Swap	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 × 3 + 2 + 1 = 10 comparisons, 6 swaps in this textbook method.

Pass	List after pass	Fixed at the front
Start	8, 4, 1, 9, 3	—
1	1, 8, 4, 9, 3	1
2	1, 3, 8, 9, 4	1, 3
3	1, 3, 4, 9, 8	1, 3, 4
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Selection Sort in Python (follows the textbook pseudocode):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(outer + 1, n):
        if data[outer] > data[inner]:
            temp = data[outer]
            data[outer] = data[inner]
            data[inner] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

After pass 1 : [1, 8, 4, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]

WORKED EXAMPLE

The "classic" Selection Sort (one swap per pass). Many books first find the position of the smallest value in the unsorted part, and swap only once at the end of each pass. The result is the same; only the swaps are fewer.

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    smallest = outer
    for inner in range(outer + 1, n):
        if data[inner] < data[smallest]:
            smallest = inner
    data[outer], data[smallest] = data[smallest], data[outer]
    print("After pass", outer + 1, ":", data)
```

OUTPUT

After pass 1 : [1, 4, 8, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]

Here only 4 swaps are made (at most N – 1). This is why Selection Sort is said to perform fewer swaps than Bubble Sort. In exams, use the textbook method unless the question asks otherwise.

Advantages	Disadvantages
Very easy to code.	Time grows very fast with list size (nested loops, about N² comparisons).
Performs fewer swaps overall compared to Bubble Sort.	Must finish every single loop — it is "blind" to an already sorted list.
Does not need extra memory.	Not stable (it might change the relative positions of equal items).

WORKED EXAMPLE

Why "not stable"? Take two equal marks, S₉ (Ali) and S₉ (Bilal), in the list S₉, S₉. 2. Pass 1 compares S₉ with S₉ (no swap), then S₉ with 2 → 2, S₉, S₉. Now Bilal is before Ali, although Ali came first originally. Bubble Sort never does this, because it swaps only neighbours that are strictly out of order.

COMMON MISTAKE

In the textbook method, the comparison is always with the **fixed position** DATA[K] (Pos1 in pass 1, Pos2 in pass 2), not with the neighbour. Students often mix this up with Bubble Sort and compare neighbours. Also, after a swap, the new value at Pos K is used for the next comparison.

آرڈو میں سمجھیں

سرکیٹنگ سارٹ (Selection Sort) میں فہرست کی غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔ ہر بائیں میں ایک مفردہ جگہ (Position) کا موازنہ اس کی بعد والی تمام عناصر سے ہوتا ہے، اس کا کوڑ آسان ہے اور بیل سارٹ سے کم تبادلے (Swaps) ہوتے ہیں، لیکن یہ مستحکم (Stable) نہیں اور ترتیب شدہ فہرست پر بھی بوجہ لوپ چلاتا ہے۔

EXAM TIP

Typical questions: "Sort ... using Selection Sort, showing all steps", "Differentiate between Bubble Sort and Selection Sort" and "Why is Selection Sort not stable?" In the difference question mention: what is compared (neighbours vs fixed position), where the inner loop starts (0 vs outer+1), where items settle (end vs front), stability, and number of swaps.

CHECK YOURSELF

1. In Selection Sort, where does the inner loop start?
2. Using the textbook method, show 7, 2, 5 after pass 1.
3. Which of the two sorts is stable?

Answers: 1. At outer + 1 (PTR = K + 1). 2. Pos1 vs Pos2: 7 > 2 swap → 2, 7, 5; Pos1 vs Pos3: 2 > 5 no → 2, 7, 5. 3. Bubble Sort.

2

Next lecture: 2.4.2 Searching Algorithms: Linear Search